
08 - Terraform Locals

1. What Are Locals in Terraform?

Locals in Terraform are values you define once and reuse multiple times inside your module or configuration.

They behave like **variables**, but with one major difference:

✓ **Variables** → come from outside (user input)

✓ **Locals** → used internally (Terraform logic)

Locals help you avoid repeating the same values across multiple resources and make your code cleaner, DRY (Don't Repeat Yourself), and easier to maintain.

2. Why Do We Use Locals?

Locals solve several real-world problems:

★ Problem 1: Repeated Values Everywhere

Without locals, you might write:

```
tags = {  
  Project = var.project  
  Managed = "terraform"  
}
```

You must repeat this for:

- VPC
- Subnets
- NAT gateway
- Route tables
- IGW

If you ever change "Managed", you update **10 places**.

Problem 2: Naming conventions

Without locals, naming resources becomes messy and inconsistent.

Problem 3: Code becomes longer, harder to maintain

Your `main.tf` becomes filled with repeated blocks.

Locals solve all these problems.

You define something once → use everywhere.

3. How Locals Work

Step 1: Define locals (usually in locals.tf)

```
locals {  
  common_tags = {  
    Project = var.project  
    Managed = "terraform"  
  }  
}
```

Step 2: Use locals inside resources

```
tags = merge(  
  local.common_tags,  
  { Name = "${var.project}-vpc" }  
)
```

Terraform automatically merges:

```
Project = "complete"  
Managed = "terraform"  
Name = "complete-vpc"
```

◆ 4. Locals vs Variables — Simple Comparison

Feature	Variables	Locals
Source	External input	Internal only
Overridable	Yes	No
?		

Purpose	User-configurable values	Reusable internal logic
Example	region, project	tags, naming patterns

◆ 5. Locals in Real-World Terraform Projects

In production, companies ALWAYS use locals for:

✓ Tags

```
locals {
  common_tags = {
    Environment = var.environment
    Owner       = "DevOps-Team"
    Managed     = "terraform"
  }
}
```

✓ Naming conventions

```
locals {
  name_prefix = "${var.project}-${var.environment}"
}
```

✓ Calculations

```
locals {
  subnet_cidrs = cidrsubnets(var.vpc_cidr, 4, 4)
}
```

✓ Conditional logic

```
locals {  
  enable_nat = var.environment == "prod"  
}
```

✓ Resource loops

```
locals {  
  public_subnets = {  
    "sub1" = "10.0.1.0/24"  
    "sub2" = "10.0.2.0/24"  
  }  
}
```

◆ 6. How Locals Improve Your VPC Project

Before locals, your tag block looked like this:

```
tags = {  
  Name    = "complete-public-subnet"  
  Project = var.project  
  Managed = "terraform"  
}
```

You repeated it 8+ times.

After locals:

You define once:

```
locals {  
  common_tags = {
```

```
Project = var.project
Managed = "terraform"
}
}
```

Then use:

```
tags = merge(local.common_tags, { Name = "${var.project}-public-subnet" })
```

◆ 7. Benefits of Using Locals

★ 1. Eliminates Repetition

You never repeat tags in every resource again.

★ 2. Centralized Control

To update tags:

Update locals → auto-updates everywhere.

★ 3. Cleaner main.tf

Your resources are shorter and readable.

★ 4. Professional Infrastructure Code

Locals are a Terraform best practice followed by:

- AWS
- Google Cloud

- Netflix
- PayPal
- Every DevOps / SRE team

★ 5. Reduces Human Error

If you miss updating 1 tag → inconsistent resources.
Locals prevent this.

◆ 8. Best Practices for Locals

✓ Use locals for:

- common tags
- naming standards
- computed values
- complex expressions
- reusable logic

✓ Keep locals in a separate file (**locals.tf**)

It keeps your project clean.

✓ Don't overuse locals

If a value should come from the user → use **variable**, not **local**.

✓ Use meaningful names

Good:

local.common_tags

local.name_prefix

Bad:

local.x

local.data1

◆ 9. Full Example of Locals in Your VPC Project

locals.tf

```
locals {  
  common_tags = {  
    Project = var.project  
    Managed = "terraform"  
  }  
}
```

Example usage in main.tf

```
tags = merge(  
  local.common_tags,  
  { Name = "${var.project}-private-rt" }  
)
```

◆ 10. Terraform Commands to View Locals

You can test locals with:

terraform console

Example:

```
> local.common_tags
```

Output:

```
{  
  "Managed" = "terraform"  
  "Project" = "complete"  
}
```

Final Summary: Why Locals Are Important

Locals:

- reduce duplication
- simplify your code
- centralize logic
- enforce consistency
- improve readability
- follow best DevOps practices

Variables + locals together make your Terraform:

- ✓ Professional
 - ✓ Scalable
 - ✓ Maintainable
 - ✓ Production-ready
-